CS FUNDAMENTALS

Understanding Caching

How computers remember things so they don't have to repeat themselves

5 Lessons | 12 Concepts | ∞ Analogies

What is Caching?

THE CAKE ANALOGY

Imagine you're a chef. Every time someone orders a chocolate cake, you could bake one from scratch — taking 45 minutes.

OR... you could keep one ready on the counter. Someone orders? Serve instantly.

Caching is keeping that cake on the counter.

Cache

A temporary storage layer that saves results of expensive operations so future requests can be served faster.

- Serve data in microseconds instead of seconds
- Reduce costly database or API calls
- Scale to millions of users without extra load

Cache Hit vs Cache Miss

CACHE HIT

Data found in cache

App → Cache → ✓ User

Fast! The result is in memory.
No trip to the database needed.

Latency: ~1ms
Database calls: 0

VS

CACHE MISS

Data NOT in cache

App → Cache → DB → Save

Slower. Must fetch from source.
Then stores the result in cache
for next time.

Latency: ~100-1000ms
Database calls: 1

Goal: Maximize hits. A 90% hit rate means only 1 in 10 requests ever touches the database.

TTL & Cache Eviction

⏱ TTL — Time To Live

Every cached item has an expiry time. After expiry, the next request fetches fresh data and repopulates the cache.

Example: TTL = 60 seconds

68% remaining

TTL expires at 60s → evicted

Eviction Policies

LRU — Least Recently Used

Evict the item that hasn't been accessed for the longest time. Most common strategy.

LFU — Least Frequently Used

Evict the least popular item overall, regardless of when last used.

FIFO — First In, First Out

Evict the oldest item inserted, like a queue. Simple but often suboptimal.

Where Do Caches Live?

CPU Cache (L1/L2/L3)
Stores recently used CPU instructions. Fastest storage on Earth.
< 1ns

Browser Cache
CSS, JS, images cached locally so pages load instantly on revisit.
~5ms

CDN Cache
Content on edge servers worldwide. Netflix, YouTube rely on this.
~20ms

App Cache (Redis)
In-memory store between app and database. Most impactful layer.
~1ms

Database
Source of truth. Always slowest — we cache to avoid hitting it.
~100ms

Cache Invalidation

> *"There are only two hard things in Computer Science: **cache invalidation** and **naming things."*** — Phil Karlton

✍ Write-Through

Consistent

Update cache AND database at the same time. Always consistent, writes are slightly slower.

📝 Write-Behind

Faster

Update cache first, sync to database later asynchronously. Faster writes, risk of data loss.

🔍 Cache-Aside

Most Common

App checks cache; on miss, fetches from DB and populates cache. Most commonly used pattern.

💥 Invalidate on Write

Simple

When source data changes, delete the cache entry. Next read fetches fresh and repopulates.

KEY TAKE-AWAYS

Remember these
5 core concepts

01 Caching = Smart Memory

Store expensive results; reuse them fast. Like keeping the cake on the counter instead of baking each time.

02 Maximize Hit Rate

Every cache miss is a database trip. A 90%+ hit rate is the target for high-performance systems.

03 Everything Expires

TTL keeps data fresh. Stale caches are bugs in disguise. Always design around expiry.

04 Caches Are Layered

CPU → Browser → CDN → App → DB. Each layer trades capacity for speed.

05 Invalidation Is Hard

The hardest problem: keeping cache and source in sync. Cache-aside and write-through are your go-to strategies.

REAL-WORLD CASE STUDIES

Caching in the Real World

How Amazon, YouTube, CNN & Banks use caching to serve millions of users without breaking.

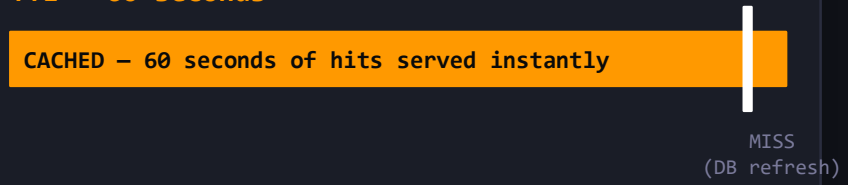
Amazon YouTube CNN Banks

Amazon — Product Page Caching

The Scenario

An iPhone listing has price, description, images, reviews. Thousands view it every minute. The data barely changes mid-day.

TTL = 60 seconds



Only 1 DB query per minute per product, regardless of how many people are viewing it simultaneously

At Black Friday: millions of hits/min, DB sees ~1,000 queries/min.

99.9%
Hit Rate
1 DB call per 60s per product

60s
TTL
acceptable staleness for product data

1000×
Less DB Load
on Black Friday vs no caching

Pattern: TTL-based caching — accept slight staleness for massive scale

YouTube — Split Caching Strategy

The Scenario

Search "caching explained" and YouTube returns titles, thumbnails, view counts, and channel names. Not all data ages the same way.

DATA	TTL	WHY
Title + Thumbnail	Hours	Almost never changes
Channel name	Hours	Rarely changes
View count	Minutes	Changes constantly
Comments	Seconds	Live activity

This is why view counts look frozen at a round number — you're seeing a cached snapshot.

Hours

TTL for thumbnails
set-and-forget for static content

Minutes

TTL for view count
balance freshness vs DB load

2 Caches

Different strategies
hot data vs live data handled separately

Pattern: Different TTLs for different data — not all data ages equally

CNN — Homepage HTML Caching

The Scenario

CNN's homepage shows the top 10 headlines. Millions load it every hour. The page only changes when editors publish something new.

The Math:

Without cache
1,000,000 DB queries/hr

With 30s TTL
120 DB queries/hr

Tradeoff: a visitor might see a headline 29 seconds stale. Totally acceptable.

8,333×
Fewer DB queries
30s TTL on 1M visitors/hr

30s
Max staleness
editors accept this tradeoff

99.997%
Hit Rate
near-perfect for mostly-static content

Pattern: Whole-page caching — even 30s of staleness saves millions of queries

Banks — Exchange Rate Caching

The Scenario

Currency rates (USD → EUR) update every few seconds on trading floors, but your banking app doesn't need millisecond precision. Thousands check rates simultaneously.

Request Flow:

- 1,000 users check rate
- All served from cache
- 1 API call per 10s

Without this: every user triggers a paid API call with rate limits.

10s
TTL for exchange rate
precise enough for consumers

$0
Extra API cost
1 call per 10s vs 1 per user

99.9%
Hit Rate
thousands served per single fetch

Pattern: Rate-limit shield — cache protects expensive external APIs from overload

THE GOLDEN FORMULA

What makes data
a great cache candidate?

Read often + Written rarely

COMPANY	DATA	FREQUENCY	HIT RATE
Twitter Celebrity profiles	Read: millions/day Written: rarely		~99%
Amazon Product listings	Read: thousands/min Written: daily		~99.9%
YouTube Titles & thumbnails	Read: billions/day Written: once		~99.9%
CNN Homepage HTML	Read: millions/hr Written: hourly		~99.997%
Banks Exchange rates	Read: thousands/min Written: 10s		~99.9%

Every example shares the same trait: data read far more often than it is written.

REMOTE DICTIONARY SERVER

Redis

The In-Memory Powerhouse

How Redis works, where it fits in the cache hierarchy, and why every major tech company depends on it.

```
127.0.0.1:6379> SET greeting "Hello Redis"
OK
```

~1ms	100k+	RAM	#1
Response time	Ops/second	Storage layer	Most-loved DB tool

What is Redis?

The Core Idea

Redis stores data entirely in RAM — no disk reads, no spinning drives. When your app asks for something, Redis hands it back in under a millisecond.

redis-cli

```
SET user:42  "Alice"
# → OK

GET user:42
# → "Alice"  (~0.3ms)
```

Lives in RAM

Not on disk. Every key-value pair sits in your server's memory for instant access.

Key-Value Store

Data is stored and retrieved by key. Simple, fast, and predictable at any scale.

You Control It

Unlike L1/L2/L3 CPU cache (automatic), YOU decide what goes in Redis, how long it stays, and when it expires.

Where Redis Sits in the Stack

Redis is application-level cache — between your code and your database. Much slower than CPU cache, but 100× faster than a database query.

L1 Cache	Inside CPU core	Hardware (automatic)	~1 ns
L2 Cache	Attached to CPU core	Hardware (automatic)	~5 ns
L3 Cache	Shared across cores	Hardware (automatic)	~30 ns
Redis	Your server RAM	YOU (explicit code)	~1 ms
Database	SSD / Disk	YOU (every query)	~100 ms

YO

CPU caches: automatic, invisible, nanoseconds

Redis: explicit, intentional, milliseconds

TTL — Making Data Expire Automatically

Set a Time To Live on any key. Redis deletes it automatically when it expires. No cron jobs. No cleanup code.

ttl-examples.py

```python
import redis

r = redis.Redis(host='localhost')

# Set with TTL = 60 seconds
r.set('rate:user:42', 1, ex=60)

# Check remaining TTL
r.ttl('rate:user:42')
# → 47  (13 seconds passed)

# After 60s: key is gone
r.get('rate:user:42')
# → None
```

TTL Timeline (60s key)

Common TTL Values:

300s	User session token
60s	Product page cache
10s	Exchange rate cache
1s	Rate limit counter

The Cache-Aside Pattern

The most common Redis pattern. Your app checks Redis first. On miss, fetch from DB and populate Redis for next time.

cache_aside.py

```
def get_user(user_id):
    key = f'user:{user_id}'

    # 1. Check Redis
    cached = r.get(key)
    if cached:
        return json.loads(cached)  # HIT

    # 2. Cache miss — query DB
    user = db.query('SELECT * FROM users'
    '  WHERE id=?'
, user_id)

    # 3. Store in Redis (TTL=5min)
    r.set(key, json.dumps(user), ex=300)
    return user  # MISS (but cached now)
```

1. App requests user:42
2. Check Redis first
3a. HIT → return instantly (~1ms)
3b. MISS → query database (~100ms)
4. Store result in Redis

Why Not Just Cache in a Variable?

In-process memory variables are fast — but they break the moment you run more than one server.

✗ Without Redis

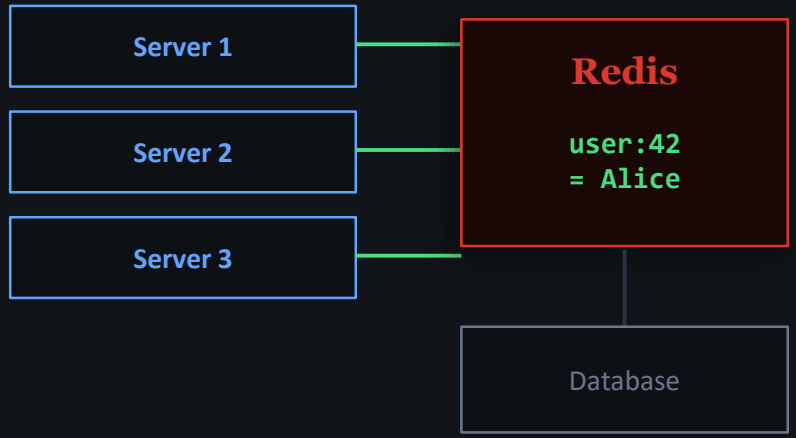
Server 1
local cache:
`user:42 = Alice`

Server 2
local cache:
`user:42 = ???`

Server 3
local cache:
`user:42 = OLD DATA`

Each server has its own memory.
User updates on Server 1 are invisible to others.
Inconsistent data. Bugs. Pain.

✓ With Redis



All servers read from ONE Redis.
Always consistent. Alice updates once, everyone sees it instantly.

What Redis Is Actually Used For

Redis is not just a cache — it's a multi-tool. Here's how major companies use it beyond simple caching.

Rate Limiting

GitHub, Stripe, Twitter

Count API calls per user per second. Expire automatically. Block abusers without touching DB.

```
INCR calls:user:42
EXPIRE calls:user:42 60
```

Session Storage

Shopify, Discord

Store login sessions in Redis. Fast auth check on every request without hitting the database.

```
SET sess:abc123
"{user_id:42}"
EXPIRE sess:abc123 3600
```

Leaderboards

Steam, Roblox, games

Sorted Sets rank millions of players in real-time. ZRANK returns position instantly.

```
ZADD scores 9500 'Alice'
ZRANK scores 'Alice' → 1
```

Job Queues

Uber, Airbnb, Netflix

Push background tasks to a Redis list. Worker processes pop and execute them asynchronously.

```
LPUSH jobs:email
'{to:alice}'
BRPOP jobs:email 0
```

Pub/Sub Messaging

Slack, WhatsApp

Publish events to channels. Subscribers receive them instantly. Powers real-time notifications.

```
PUBLISH chat:room1
'Hello!'
SUBSCRIBE chat:room1
```

Atomic Counters

YouTube, Reddit

INCR is atomic — no race conditions. Count views, votes, or likes safely across thousands of servers.

```
INCR views:video:99
→ 1,000,001 (instant)
```

REDIS IN 5 POINTS

Everything you need to remember

```
redis> PING
PONG
```

01 RAM-first storage

Redis lives entirely in memory. No disk reads. That's why it's ~1ms vs ~100ms for a database.

02 Application-level cache

Sits between your app and database. Slower than CPU cache, but 100× faster than DB queries.

03 You control everything

Unlike L1/L2/L3 (automatic), you explicitly choose what goes in Redis, how long it lives, and when it's gone.

04 Shared across all servers

One Redis instance serves all your app servers consistently. No stale local variables per machine.

05 More than just caching

Rate limiting, sessions, leaderboards, queues, pub/sub — Redis is a Swiss Army knife

The golden rule: if it's read often and changes rarely, put it in Redis.

Redis Data Types

One type per slide · Big code · One example each

String | Hash | List | Set | Sorted Set | Pub/Sub

String

Store anything. Get it back instantly.

```
SET  name  "Alice"
GET  name  →  "Alice"

INCR views →  1001
```

Use for: caching · counters · session tokens

Hash

An object stored field by field.

user:42	
name	"Alice"
age	"30"
city	"NYC"

```
HSET user:42
  name  "Alice"
  age   "30"

HGET user:42  name
→  "Alice"
```

Use for: user profiles · update one field without rewriting all

List

Ordered sequence. Push or pop from either end.

feed:alice

- **post_107**
- post_106
- post_105

```
LPUSH feed  "post_107"

LRANGE feed  0  9
→  [107, 106, 105]

BRPOP jobs  0
```

Use for: activity feeds · job queues · BRPOP blocks until work arrives

Set

Unique values only. Duplicates silently ignored.

Alice	Bob
u:7	u:7
u:8	u:8
u:9	u:99

SINTER → u:7, u:8

```
SADD  likes  "u:1"  "u:7"
SADD  likes  "u:1"
# duplicate ignored ✓

SCARD  likes  →  2

SINTER A  B  →  mutual
```

Use for: likes · tags · SINTER finds mutual friends in one command

Sorted Set

Every member has a score. Always ranked automatically.

leaderboard

#1	Charlie	12000
#2	Alice	9500
#3	Bob	8800

```
ZADD lb  12000  "Charlie"

ZREVRANGE lb  0  2
→ [Charlie, Alice]

ZREVRANK lb  "Alice"
→ 1   # 2nd place
```

Use for: leaderboards · rankings · instant rank lookup in O(log n)

Pub / Sub

Publish once. Every subscriber gets it in < 1ms.

```
SUBSCRIBE   channel:news
PUBLISH     channel:news   "Hello!"
→  (integer) 3    # 3 subscribers got it
```

Use for: real-time chat · live notifications · no message storage

Quick Reference

String	SET / GET / INCR	Cache any value · counters · session tokens
Hash	HSET / HGET / HGETALL	Objects · update one field at a time
List	LPUSH / RPUSH / BRPOP	Feeds · queues · blocks waiting for work
Set	SADD / SCARD / SINTER	Unique items · find overlap between groups
Sorted Set	ZADD / ZREVRANK / ZINCRBY	Leaderboards · rankings by score
Pub/Sub	PUBLISH / SUBSCRIBE	Real-time events · no storage needed

IN-MEMORY CACHE vs REDIS

In-Memory Variable

dict/HashMap inside your app process

```
cache = {}
cache["user:42"] = "Alice"
```

- ✓ Fastest possible (~ns)
- ✓ No extra setup needed
- ✓ Zero network overhead
- ✗ Lost on restart
- ✗ Each server has own copy
- ✗ Not shared across servers

Use when: 1 server · speed critical · small temp data

VS

Redis

Dedicated in-memory server — external to your app

```
r = redis.Redis(host='localhost')
r.set("user:42", "Alice")
```

- ✓ Shared by ALL servers
- ✓ Survives app restarts
- ✓ TTL, rich data types
- ✗ ~1ms network round-trip
- ✗ Extra server to manage
- ✗ You write explicit code

Use when: multiple servers · shared state · TTL needed